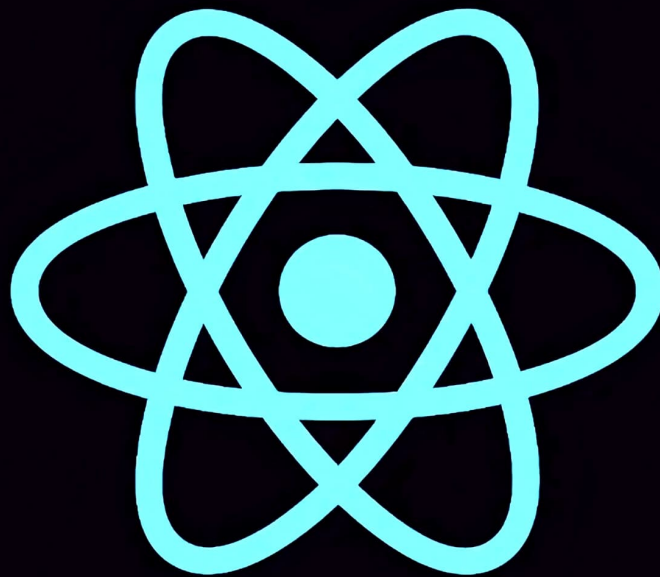


Advanced React Concepts

Everyone should know



1. Higher-Order Components (HOC)

A function that takes a component and returns a new component with additional props or functionality.



```
const withLogger = (WrappedComponent) => {  
  return (props) => {  
    console.log('Props:', props);  
    return <WrappedComponent {...props} />;  
  };  
};  
  
const Hello = ({ name }) => <h1>Hello {name}</h1>;  
const HelloWithLogger = withLogger(Hello);  
  
// Usage  
<HelloWithLogger name="John" />
```

2. Render Props

A technique where a component uses a function as a prop to determine what to render.



```
const MouseTracker = ({ render }) => {  
  const [pos, setPos] = React.useState({ x: 0, y: 0 });  
  return (  
    <div onMouseMove={(e) => setPos({ x: e.clientX, y: e.clientY })}>  
      {render(pos)}  
    </div>  
  );  
};  
  
// Usage  
<MouseTracker render={({ x, y }) => <p>Mouse: {x}, {y}</p>} />
```

3. Context API with useContext

Allows sharing state globally without prop drilling.



```
const ThemeContext = React.createContext('light');

const ThemedButton = () => {
  const theme = React.useContext(ThemeContext);
  return <button style={{ background: theme === 'dark' ? '#333' :
  '#fff' }}>Click</button>;
};

<ThemeContext.Provider value="dark">
  <ThemedButton />
</ThemeContext.Provider>
```

4. useCallback

Memoizes a function to prevent unnecessary re-renders.

```
const Counter = () => {  
  const [count, setCount] = React.useState(0);  
  const increment = React.useCallback(() => setCount(c => c + 1),  
    []);  
  return <button onClick={increment}>{count}</button>;  
};
```

5. useMemo

Memoizes a computed value to optimize performance.



```
const ExpensiveCalc = ({ num }) => {  
  const result = React.useMemo(() => num * 2, [num]);  
  return <p>{result}</p>;  
};
```

6. Code Splitting with React.lazy & Suspense

Loads components only when needed (lazy loading)



```
const LazyComponent = React.lazy(() => import('./LazyComponent'));  
  
<Suspense fallback={<p>Loading...</p>}>  
  <LazyComponent />  
</Suspense>
```